

AI Engineer Domain-Based Capstone Projects

Client-style group projects covering GenAI, Agentic AI, Cloud, Deployment, MLOps and Responsible AI

For 6 groups | Domains: Banking, Healthcare, Telecom, Automotive, Enterprise Business and Retail

Objective

Each group will collaboratively build a domain-specific AI Engineer solution using Git and GitHub. Every project must include a working AI demo plus engineering baseline: Streamlit UI, FastAPI backend, Dockerfile, architecture, cloud deployment design, monitoring/logging, LLMops/MLOps plan and responsible AI controls.

Common Mandatory Requirements for All Groups

- GitHub repository with clean README, setup steps, run commands and contributor details.
- Proper Git collaboration: feature branches, meaningful commits, pull requests and code review comments.
- Streamlit UI for end-user demo.
- FastAPI backend endpoint for the main workflow.
- Dockerfile or clear containerization plan.
- Architecture diagram showing UI, API, AI workflow, data layer and external services.
- Cloud deployment design using Azure OpenAI, AWS Bedrock or Google Vertex AI concept.
- Monitoring and logging strategy: query count, latency, errors, retrieved sources and token/cost tracking concept.
- LLMops/MLOps strategy: prompt versioning, evaluation set, feedback loop and regression checks.
- Responsible AI controls: citations, answer-not-found behavior, privacy, access-control design and human review for high-risk outputs.
- Sample or synthetic data only. Do not use confidential client or participant data.
- Final presentation covering problem statement, architecture, implementation, demo, challenges and future scope.

Project Summary

Group	Domain	Project Topic	Business Scenario	Core Engineering Focus	Participants Email IDs
Group 1	Banking & Financial Services	Intelligent Loan Advisor and Policy Assistant	Banks need AI assistants that help relationship managers and operations teams answer loan eligibility, required document, product policy, repayment and collection guideline questions from approved policy documents.	GenAI, RAG, policy retrieval, eligibility tool, citations, FastAPI, Streamlit, Docker	akashkumarmuratlal.yadav@wipro.com akshay.singh1@wipro.com mohammad.arshad2@wipro.com suyogvijayyadav.yadav@wipro.com babuprasanth.s@wipro.com manisankar.k1@wipro.com
Group 2	Healthcare	Clinical Knowledge and Patient Education Assistant	Healthcare teams need safe assistants that search approved guidelines, insurance policies and patient education content, then generate simple, grounded explanations for staff or patient-service teams.	GenAI, Healthcare RAG, summarization, safety guardrails, FastAPI, Streamlit, Docker	aparna.jha1@wipro.com shivang.malhotra@wipro.com medha.pragya@wipro.com nandavardhan.doodala@wipro.com saswata.halder@wipro.com
Group 3	Telecommunications / Communication	Telecom Support Agent with Network Issue Triage and Escalation	Telecom customers report network issues, billing concerns, roaming failures and SIM activation problems. Support teams need AI assistance to classify issues, suggest	Agentic AI, tool calling, LangGraph workflow, support memory, FastAPI, Streamlit, Docker	abhishek.kumar57@wipro.com asmit.singh@wipro.com kartik.rudragoudar@wipro.com kmpallavi.srivastava@wipro.com trupti.annasahebkhondhare@wipro.com

			troubleshooting steps and escalate unresolved cases.		
Group 4	Automotive	Vehicle Diagnostics and Service Recommendation Assistant	Automotive service teams need help interpreting diagnostic codes, manuals, service procedures and maintenance recommendations for vehicles using approved service documents.	GenAI, RAG, troubleshooting workflow, service recommendation agent, FastAPI, Streamlit, Docker	satyam.maurya@wipro.com rameshsaheb.jadhav@wipro.com kartik.angrish@wipro.com syedmohammad.baaqui@wipro.com pratheep.p@wipro.com
Group 5	Enterprise Business / Market Intelligence	Multi-Agent Market Research and Executive Reporting Platform	Business teams need market research, competitor analysis, industry trend summaries and executive reports created from approved internal or sample research documents.	Agentic AI, LangGraph, multi-agent orchestration, verifier agent, report generation, FastAPI, Docker	dharshan.n06@wipro.com rahul.sharma8@wipro.com smsohel.biswas@wipro.com rk.priyan@wipro.com saran.r39@wipro.com chintan.akalusrinath@wipro.com
Group 6	Retail / E-Commerce	Product Recommendation and Customer Insights Assistant	Retail and e-commerce teams need AI assistants that understand product catalogs, compare products, answer customer questions and generate customer-insight summaries for campaigns or support teams.	GenAI, RAG, recommendation logic, tool calling, customer insights, FastAPI, Streamlit, Docker	rudranil.pal@wipro.com himanshushekhhar.pollai@wipro.com anshul.bhati1@wipro.com vunnamsai.koushik@wipro.com suryalavakusa.chandina@wipro.com divyansh.raiswal@wipro.com

Group 1: Banking & Financial Services - Intelligent Loan Advisor and Policy Assistant

Client-style need	Clients in BFS often want secure knowledge assistants that reduce manual policy search and provide grounded, auditable responses for loan operations.
Problem statement	Build a Banking GenAI assistant that answers loan policy questions, checks basic eligibility using a simulated rule/tool, and provides source-aware answers using sample loan and banking policy documents.
Core AI components	GenAI answer generation; RAG over banking policy/SOP documents; Tool calling for simple loan eligibility check; Hybrid search for exact policy codes or product names; Answer-not-found behavior with citations
Core engineering components	Streamlit UI for user questions; FastAPI endpoint for ask and eligibility-check workflows; Vector DB using FAISS/Chroma; Dockerfile and environment-based configuration; Logging of query, latency and retrieved source IDs

Cloud, Deployment and Security Expectations

- Azure OpenAI or equivalent cloud LLM design
- Cloud deployment architecture with UI, API, vector DB and LLM layer
- Secrets in environment variables or vault design
- Cost estimation based on token usage and query volume

MLOps / LLMOps Expectations

- Prompt versioning
- Golden Q&A evaluation set
- Retrieval quality checks
- Feedback capture for wrong or unhelpful answers
- Monitoring dashboard concept

Minimum Features

- Upload/sample loan policy documents
- Chunk, embed and retrieve relevant policy sections
- Ask loan/product/policy questions and show citations
- Simulated eligibility check tool based on synthetic inputs
- Fallback when answer is not found in source documents

Expected Deliverables

- Working Banking RAG demo
- FastAPI API contract
- Architecture and deployment diagram
- Synthetic banking documents and test questions
- README, Dockerfile, GitHub PR evidence and final presentation

Participants

- akashkumarmuratlal.yadav@wipro.com
- akshay.singh1@wipro.com
- mohammad.arshad2@wipro.com
- suyogvijayyadav.yadav@wipro.com
- babuprasanth.s@wipro.com
- manisankar.k1@wipro.com

Group 2: Healthcare - Clinical Knowledge and Patient Education Assistant

Client-style need	Healthcare clients expect controlled, source-grounded AI assistants that avoid unsupported medical claims and use only approved content.
Problem statement	Build a Healthcare GenAI assistant using synthetic/sample healthcare policies and patient education documents. The assistant should answer guideline questions, summarize patient education content and clearly state when source evidence is insufficient.
Core AI components	GenAI explanation generation; RAG over synthetic healthcare guideline/policy documents; Prompt engineering for simple patient-friendly summaries; Grounded answer generation with source citations; Safety-focused unsupported-answer handling
Core engineering components	Streamlit UI for staff questions and patient education generation; FastAPI backend for question-answer and summarize endpoints; Vector DB using FAISS/Chroma; Dockerfile and structured repository; Basic logs for query, source and response time

Cloud, Deployment and Security Expectations

- Azure OpenAI / Vertex AI / AWS Bedrock design option
- Cloud architecture showing UI, API, vector DB and model endpoint
- Data privacy controls for synthetic-only demonstration
- Cost and latency monitoring strategy

MLOps / LLMOps Expectations

- Prompt/version tracking
- Evaluation questions for guideline accuracy
- Human review checklist for high-risk answers
- User feedback capture
- Monitoring for hallucination and answer-not-found cases

Minimum Features

- Use synthetic healthcare and insurance documents
- Ask guideline, eligibility or education questions
- Generate simple patient education summary from approved text
- Show source document and section/page reference
- Display safety disclaimer and answer-not-found response

Expected Deliverables

- Working Healthcare RAG demo
- FastAPI endpoint and Streamlit UI
- Responsible AI checklist
- Synthetic healthcare documents and sample questions
- Cloud/deployment diagram, README, Dockerfile and final presentation

Participants

- aparna.jha1@wipro.com
- shivang.malhotra@wipro.com
- medha.pragya@wipro.com
- nandavardhan.doodala@wipro.com
- saswata.halder@wipro.com

Group 3: Telecommunications / Communication - Telecom Support Agent with Network Issue Triage and Escalation

Client-style need	Telecom clients often ask for support automation that can classify tickets, retrieve troubleshooting guidance and route complex issues to the right support queue.
Problem statement	Build an Agentic Telecom Support Assistant that classifies customer problems, retrieves relevant troubleshooting steps, simulates ticket creation and routes unresolved or high-priority issues to escalation.
Core AI components	Intent classification using LLM prompting; RAG over telecom FAQ/troubleshooting documents; Tool calling for simulated ticket creation; LangGraph workflow for triage, resolution and escalation; Conversation memory for follow-up questions
Core engineering components	Streamlit support-agent UI; FastAPI endpoint for classify, ask and create-ticket simulation; SQLite/JSON ticket store; Dockerfile and clean repository structure; Logging of issue category, priority and escalation status

Cloud, Deployment and Security Expectations

- Cloud deployment architecture for support chatbot
- API gateway/authentication design
- Managed LLM endpoint design
- Monitoring and alerting strategy for failed or escalated tickets

MLOps / LLMOps Expectations

- Prompt versioning for classification
- Evaluation set for intent and priority classification
- Ticket resolution feedback loop
- Trace review for tool calls
- Cost and latency tracking

Minimum Features

- Telecom FAQ and troubleshooting knowledge base
- Customer issue classification by category and priority
- Suggested troubleshooting steps
- Simulated ticket creation and escalation workflow
- Conversation memory for follow-up interaction

Expected Deliverables

- Working telecom support agent demo
- LangGraph or workflow diagram
- Ticket payload examples
- FastAPI and Streamlit implementation
- README, Dockerfile, GitHub PR evidence and final presentation

Participants

- abhishek.kumar57@wipro.com
- asmit.singh@wipro.com
- kartik.rudragoudar@wipro.com
- kmpallavi.srivastava@wipro.com
- trupti.annasahebkhondhare@wipro.com

Group 4: Automotive - Vehicle Diagnostics and Service Recommendation Assistant

Client-style need	Automotive clients can use AI assistants to speed up service lookup, diagnostic-code explanation and guided maintenance recommendations from service manuals.
Problem statement	Build an Automotive GenAI assistant that accepts a diagnostic code or symptom, retrieves relevant service-manual content, suggests possible causes and recommends next service steps with source references.
Core AI components	RAG over synthetic service manuals and diagnostic documents; GenAI explanation of diagnostic codes and symptoms; Tool-like workflow for service recommendation selection; Grounded answer generation with citations; Safety/fallback response when source data is unavailable
Core engineering components	Streamlit UI for diagnostic query and service output; FastAPI endpoint for diagnose workflow; Vector DB using FAISS/Chroma; Dockerfile and configuration files; Logging of diagnostic code, retrieved source and response

Cloud, Deployment and Security Expectations

- Cloud architecture for service center assistant
- Managed LLM endpoint design
- Deployment diagram with API, UI and vector store
- Cost and latency considerations for service-center usage

MLOps / LLMOps Expectations

- Evaluation set for diagnostic-code retrieval
- Prompt version control
- Feedback capture from service engineer review
- Monitoring for unresolved queries and low-confidence answers
- Retrieval performance tracking

Minimum Features

- Use synthetic automotive manuals and diagnostic code samples
- Query by diagnostic code or symptom
- Retrieve relevant manual/service sections
- Generate possible causes and recommended next steps
- Show citations and answer-not-found behavior

Expected Deliverables

- Working automotive assistant demo
- FastAPI and Streamlit implementation
- Architecture/deployment diagram
- Synthetic diagnostics dataset and test cases
- README, Dockerfile, GitHub PR evidence and final presentation

Participants

- satyam.maurya@wipro.com
- rameshsaheb.jadhav@wipro.com
- kartik.angrish@wipro.com
- syedmohammad.baaqui@wipro.com
- pratheep.p@wipro.com

Group 5: Enterprise Business / Market Intelligence - Multi-Agent Market Research and Executive Reporting Platform

Client-style need	Enterprise clients require agentic systems that can collect information, validate evidence, summarize insights and generate business-ready reports with traceability.
Problem statement	Build a multi-agent research system where a Research Agent retrieves relevant content, a Verifier Agent checks grounding, a Summarizer Agent creates insights and a Report Generator produces an executive-style report.
Core AI components	LangGraph multi-agent workflow; Research, verifier, summarizer and report generator agents; Shared state using TypedDict or similar schema; RAG over sample market/business documents; Retry loop when verification fails
Core engineering components	Streamlit or notebook-based demo UI; FastAPI endpoint for report generation request; Structured report output; Dockerfile and modular code structure; Tracing/logging of each agent step

Cloud, Deployment and Security Expectations

- Cloud deployment architecture for multi-agent service
- Managed LLM and vector DB design
- Observability/tracing design using LangSmith or logs
- Cost control through iteration limits and prompt budgeting

MLOps / LLMOps Expectations

- Prompt and workflow versioning
- Evaluation checklist for groundedness and report quality
- Human approval before final report
- Agent run monitoring and failure analysis
- Feedback loop for improving research quality

Minimum Features

- Research agent retrieves relevant sample documents
- Verifier checks whether claims are grounded in sources
- Summarizer creates concise business insights
- Report generator produces structured executive report
- Shared state and clear graph workflow orchestration

Expected Deliverables

- Working multi-agent demo
- LangGraph workflow diagram
- Sample generated report
- FastAPI endpoint or notebook workflow
- README, Dockerfile, GitHub PR evidence and final presentation

Participants

- dharshan.n06@wipro.com
- rahul.sharma8@wipro.com
- smsohel.biswas@wipro.com
- rk.priyan@wipro.com
- saran.r39@wipro.com
- chintan.akalusrinath@wipro.com

Group 6: Retail / E-Commerce - Product Recommendation and Customer Insights Assistant

Client-style need	Retail clients often ask for product discovery, product comparison, cross-sell support and customer-insight assistants using catalogs and customer interaction data.
Problem statement	Build a Retail GenAI assistant that answers product questions from a sample catalog, compares products, recommends related products using simple rules and generates customer insight summaries from synthetic interaction data.
Core AI components	RAG over product catalog and FAQ documents; GenAI product comparison and explanation; Tool calling for recommendation rule lookup; Customer insight summarization; Grounded responses with source references
Core engineering components	Streamlit shopping-assistant UI; FastAPI endpoints for ask, compare and recommend workflows; Vector DB using FAISS/Chroma; Dockerfile and GitHub repository discipline; Logging of query, recommendation and source IDs

Cloud, Deployment and Security Expectations

- Cloud architecture for e-commerce assistant
- Managed LLM endpoint and vector DB design
- API layer integration with product systems concept
- Cost and latency monitoring strategy

MLOps / LLMops Expectations

- Prompt versioning
- Evaluation set for product Q&A and recommendations
- Feedback capture for helpful/unhelpful recommendations
- Monitoring of popular queries and failed searches
- Responsible AI checklist for product claims

Minimum Features

- Use synthetic product catalog, FAQ and customer interaction data
- Product Q&A with source citations
- Product comparison summary
- Rule-based related product recommendation
- Customer insight summary for selected category or product

Expected Deliverables

- Working retail assistant demo
- FastAPI and Streamlit implementation
- Synthetic product catalog and customer data
- Architecture/deployment diagram
- README, Dockerfile, GitHub PR evidence and final presentation

Participants

- rudranil.pal@wipro.com
- himanshushekhar.pollai@wipro.com
- anshul.bhati1@wipro.com
- vunnamsai.koushik@wipro.com
- suryalavakusa.chandina@wipro.com
- divyansh.raiswal@wipro.com

Suggested Evaluation Rubric

Evaluation Area	Weightage	What to Check
Problem understanding and domain fit	10%	Clear domain problem, realistic use case and user journey
GenAI / RAG / Agentic workflow	25%	Quality of retrieval, workflow design, tool usage, structured output or multi-agent orchestration
Engineering quality	15%	Code structure, error handling, configuration, reusable components
Git and GitHub collaboration	15%	Branches, commits, pull requests, README, task ownership
UI / API / Demo readiness	15%	Working Streamlit UI, FastAPI endpoint, smooth demo and meaningful examples
Cloud, deployment and LLMOps thinking	10%	Docker, deployment architecture, monitoring, logging, prompt evaluation and cost awareness
Responsible AI and presentation	10%	Citations, privacy controls, answer-not-found behavior, clear explanation and Q&A readiness

Recommended GitHub Collaboration Discipline

- One main branch and separate feature branches for each module or contributor.
- Every feature should be merged through a pull request, even if the group is small.
- Each group should maintain issues/tasks for UI, API, AI workflow, data, Docker, testing, documentation and presentation.
- Commit messages should be meaningful, for example: add retrieval pipeline, implement FastAPI endpoint, add Streamlit UI, add Dockerfile.
- README should include project overview, domain scenario, architecture, setup, run command, API contract, contributors and demo screenshots.